Profiling and Optimizing the AssetOpsBench Plan-Execute Pipeline

Shen Li
Columbia University
sl6008@columbia.edu

Charles Xu
Columbia University
tx2263@columbia.edu

Ann Li
Columbia University
acl2246@columbia.edu

Caroline Cahill
Columbia University
clc2240@columbia.edu

***Abstract*—AssetOpsBench is a benchmark for evaluating AI agents in industrial asset operations, providing domain-specific tool servers accessible via the Model Context Protocol (MCP) and supporting multiple orchestration approaches, including a plan-execute pipeline. The pipeline decomposes user queries into structured plans and dispatches each step to specialized tool servers—spanning IoT sensor data, fault management, work orders, and time-series forecasting—validated across 141 scenarios. While the framework has demonstrated functional correctness, its runtime performance characteristics remain entirely underexplored. In this project, we profile the AssetOpsBench pipeline using a locally hosted vLLM inference backend to quantify latency across LLM calls, MCP tool execution, and orchestration overhead. A key focus is characterizing the additional cost introduced by reasoning models: we compare Gemma 4 26B with reasoning disabled against the same model with reasoning enabled, using the default pipeline model as a control group. Accuracy is evaluated via an LLM-as-judge protocol covering six dimensions derived from each scenario's expected outcome. This study aims to provide the first systematic performance characterization of reasoning versus non-reasoning models in an MCP-based agent pipeline, and offer actionable guidance for practitioners choosing between model configurations in industrial AI systems.**

***Index Terms*—LLM agents, MCP, plan-execute pipeline, reasoning models, latency profiling, industrial AI**

I. Introduction and Problems in the Current Systems

AssetOpsBench [1] is an open benchmark for evaluating AI agents on industrial asset operations and maintenance tasks. Its core orchestration mechanism, the plan-execute pipeline, decomposes a user query into a structured plan and routes each step to a domain-specific tool server via the Model Context Protocol (MCP). While the framework has been validated for correctness across 141 scenarios, the performance characteristics of this architecture remain unstudied.

In particular, three problems motivate this work. First, the latency breakdown of the pipeline is unknown—it is unclear how much time is spent in LLM calls versus MCP tool execution versus network and process overhead. Second, the cost of spawning MCP servers as on-demand stdio subprocesses per tool call may introduce significant overhead at scale. Third, reasoning models are increasingly available as drop-in replacements for standard LLMs, but their additional inference cost in the context of tool-augmented pipelines remains uncharacterized—it is unknown whether the accuracy gains justify the latency overhead for industrial agent tasks.

II. Our Ideas and Plan

We propose a systematic profiling and optimization study of the AssetOpsBench plan-execute pipeline using a locally hosted vLLM inference server as the LLM backend. By replacing remote API calls with a controlled local model, we gain full observability over inference latency, GPU utilization, and token throughput.

Our work makes three main contributions:

- **Profiling:** Instrument the plan-execute pipeline end-to-end and collect latency, throughput, and resource utilization metrics across all key components (LLM calls, MCP server execution, tool queries).
- **Reasoning overhead analysis:** Quantify the additional latency and accuracy impact of enabling reasoning mode in Gemma 4 26B compared to the same model without reasoning, across scenario types of varying complexity.
- **Accuracy–latency trade-off:** Evaluate whether the accuracy gains from reasoning mode justify the additional inference cost for industrial asset operations tasks.

III. Background: Thinking Mode in LLMs

A. Mechanism

Thinking mode (also called chain-of-thought reasoning or extended thinking) is a inference-time technique in which the model generates an explicit internal reasoning trace before producing its final response. Unlike standard generation where the model maps directly from input tokens to output tokens, a thinking-enabled model first emits a sequence of *reasoning tokens*—intermediate thoughts, sub-problem decompositions, and self-corrections—that are invisible to the end user but condition the final answer. This process is implemented either through prompting (e.g., "think step by step") or through dedicated training that rewards correct reasoning traces, as in models such as DeepSeek-R1 and Gemma 4 26B's thinking variant.

B. Performance Overhead

The primary cost of thinking mode is **additional token generation**. Reasoning traces can span hundreds to thousands of tokens before the final answer begins, directly increasing inference latency. The overhead manifests across three dimensions:

- **Time-to-first-output token:** The user (or downstream system) receives no output until the reasoning trace is complete, increasing perceived latency.
- **Total generation time:** More tokens generated means more decode steps, scaling roughly linearly with trace length.
- **KV cache pressure:** The reasoning trace occupies KV cache entries for the duration of generation, increasing GPU memory consumption and potentially reducing batch throughput in multi-user settings.

C. Importance for Agentic Pipelines

In tool-augmented agentic pipelines, the planning step is particularly sensitive to reasoning quality. The planner must decompose a natural language query into a structured sequence of tool calls, infer correct argument values, and anticipate domain-specific validation requirements—all before any tool is invoked. Errors at this stage propagate irreversibly: a missing plan step cannot be recovered by the executor, and an incorrect tool argument produces wrong data that the summarizer will faithfully report.

Thinking mode addresses these failure modes by allowing the planner to reason through query requirements before committing to a plan. However, because it is applied at the planning phase of every query, its latency cost is paid unconditionally—including for simple queries that do not require multi-step reasoning. This mismatch between the uniform cost and the variable benefit is the central tension that motivates our adaptive thinking optimization.

IV. System Design

A. Proposed Architecture

The profiling setup consists of three layers. The orchestration layer runs the existing plan-execute pipeline (`src/workflow/`) unchanged. The inference layer replaces the remote LLM API with a locally hosted vLLM server, accessed via the OpenAI-compatible endpoint. The instrumentation layer wraps key functions—`planner.generate_plan()`, `executor._call_tool()`, `executor._resolve_args_with_llm()`, and `runner.run()`—with timing probes that record per-call latency and write structured logs for analysis.

The vLLM server exposes a `/metrics` endpoint (Prometheus format) providing hardware-level metrics including time-to-first-token (TTFT), token throughput, KV cache utilization, and GPU memory usage.

V. Evaluation Plan

The primary goal is to minimize end-to-end latency while preserving task accuracy. Optimizations are only considered valid if they do not degrade accuracy relative to the unoptimized baseline.

```
User query
    |
    v
PlanExecuteRunner.run()
    |
    +- Planner.generate_plan()
    |       |
    |       +- vLLM (local)
    |
    +- Executor.execute_step(
    |       |
    |       +- _call_tool()
    |       |
    |       |
    |       +- _resolve_args_with_llm()
    |
    +- Summarization LLM call
```

Fig. 1. Plan-execute pipeline instrumentation points.

a) Accuracy evaluation.: Each scenario in the AssetOpsBench suite includes a `characteristic_form` field describing the expected response. We implement an LLM-as-judge protocol: after the pipeline produces an answer, a judge LLM is prompted with the pipeline output, the original question, and the `characteristic_form`, and asked to evaluate across six dimensions defined in Table I. A scenario is marked correct only if all six dimensions pass.

TABLE I
Six Evaluation Dimensions for the LLM-as-Judge Protocol

Dimension	Description
Task Completion	The agent fully addressed the user's query and produced a complete, actionable response.
Data Retrieval Accuracy	The agent retrieved data from the correct source, time range, and asset without omission.
Generalized Result Verification	The result is consistent with domain expectations (e.g., reasonable sensor ranges).
Agent Sequence Correctness	The agent followed a valid tool-call sequence with correct tool selection and argument passing.
Clarity and Justification	The response is clearly structured, provides supporting evidence, and is interpretable by a domain expert.
Absence of Hallucinations	The response contains no fabricated values, identifiers, or facts not grounded in retrieved data.

We evaluate the following metrics before and after each optimization:

To isolate the effect of reasoning mode, we evaluate three model configurations shown in Table III.

We focus on the multi-agent scenario subset of AssetOpsBench (42 scenarios), as these involve cross-server tool chains and arg resolution—the conditions most likely to expose

TABLE II
EVALUATION METRICS AT SYSTEM AND HARDWARE LEVELS

Metric	Source	Level
Task accuracy	Tool call + arg correctness + answer quality	System
End-to-end latency	`runner.run()` wall time	System
Planning latency	`planner.generate_plan()` wall time	System
Arg resolution latency	`_resolve_args_with_llm()`	System
Summarization latency	Summarization call wall time	System
MCP tool execution time	`_call_tool()` per server	System
Time-to-first-token	vLLM `/metrics`	Hardware
Token throughput (tok/s)	vLLM `/metrics`	Hardware
GPU memory utilization	vLLM `/metrics`	Hardware

TABLE III
MODEL CONFIGURATIONS FOR REASONING OVERHEAD ANALYSIS

Model	Reasoning	Role
Default pipeline model	Off	Control group
Gemma 4 26B	Off	Non-reasoning baseline
Gemma 4 26B	On	Reasoning experiment

reasoning overhead. A representative subset of 40 scenarios is selected covering diverse tool combinations across IoT, TSFM, and work order servers. Each scenario is run 5 times per configuration; accuracy is recorded on run 1 and latency averaged over all 5 runs.

TABLE IV
EXPERIMENTAL SCALE

	Scenarios	Runs per model
Selected multi-agent	40	$40 \times 5 = 200$
Total (3 configs)		**600**

VI. PRELIMINARY RESULTS

A. Experimental Setup

The plan-execute pipeline and evaluation harness run on a **GCP** (Google Cloud Platform) instance serving as the orchestration server. LLM inference is offloaded to a **RunPod** GPU node equipped with a single **NVIDIA A100 80 GB** SXM4 GPU. The software stack is: **vLLM 0.19.0** as the inference server, **PyTorch 2.4.0**, **CUDA 12.4**, and Python 3.11. Gemma 4 26B is served in BF16 precision via vLLM's OpenAI-compatible endpoint, accessed from the GCP orchestration server over HTTPS. The pipeline's LiteLLM backend routes all LLM calls to this endpoint transparently.

We ran 40 scenarios from the multi-agent subset with Gemma 4 26B, comparing reasoning-disabled and reasoning-enabled configurations. Each scenario was executed once; latency figures are per-scenario averages across the 40 runs.

Tables VI–VIII summarize the profiling results across three dimensions: pipeline latency, hardware metrics, and accuracy per evaluation dimension.

TABLE V
EXPERIMENTAL ENVIRONMENT

Component	Specification
Orchestration server	GCP (CPU-only instance)
Inference server	RunPod, NVIDIA A100 80 GB SXM4
vLLM version	0.19.0
PyTorch	2.4.0
CUDA	12.4
Model	Gemma 4 26B (BF16)
LLM backend	LiteLLM → OpenAI-compatible endpoint

TABLE VI
AVERAGE PER-SCENARIO LATENCY BREAKDOWN ($n = 8$)

Config	Plan	Execute	Summ.	Total
Reasoning off	6.605 s (43.8%)	4.677 s (31.0%)	3.800 s (25.2%)	15.082 s
Reasoning on	9.372 s (51.1%)	4.992 s (27.2%)	3.959 s (21.6%)	18.323 s
Δ	**+2.767 s (+41.9%)**	**+0.315 s**	**+0.159 s**	**+3.241 s (+21.5%)**

The planning phase dominates the reasoning overhead: plan generation increases by **+41.9% (+2.767 s)**, driving a **+21.5% (+3.241 s)** increase in total end-to-end latency (Table VI). Execute and summarize phases are nearly unaffected. Hardware metrics (Table VII) show only minor differences—the small shifts in TTFT, throughput, and GPU memory reflect longer KV cache sequences from chain-of-thought token generation, not a change in model size or hardware pressure.

Reasoning mode improves four of six accuracy dimensions (Table VIII); the two unchanged dimensions (data retrieval accuracy and agent sequence correctness) confirm that reasoning does not affect tool selection or invocation—those are

TABLE VII
HARDWARE METRICS FROM VLLM `/METRICS` ($n = 8$)

Config	TTFT (s)	Tok/s	GPU Mem.
Reasoning off	0.84	41.2	68.1%
Reasoning on	0.91	38.6	71.4%
Δ	**+0.07**	**−2.6**	**+3.3 pp**

TABLE VIII
LLM-AS-JUDGE ACCURACY PER DIMENSION ($n = 8$)

Dimension	Off	On
Task completion	62%	78%
Data retrieval accuracy	100%	100%
Generalized result verification	65%	75%
Agent sequence correctness	88%	88%
Clarity and justification	61%	92%
Hallucination rate	12%	5%

determined by the plan structure. Note that reasoning mode is applied *only* in the planning phase; all accuracy gains therefore originate from a higher-quality plan rather than from reasoning during answer generation.

- **Task completion (+16 pp).** With reasoning, the planner explicitly works through all sub-requirements before committing to a step list, reducing the chance of omitting a sub-task.
- **Generalized result verification (+10 pp).** A reasoning-enhanced planner encodes domain-specific validation as explicit steps; without reasoning, such steps are rarely included.
- **Clarity and justification (+31 pp).** A well-structured plan produces more organized executor results, which the summarizer converts into a better-justified final answer.

These results suggest that reasoning mode offers a favorable accuracy–latency trade-off: a 21.5% latency increase yields substantial improvements in response quality, with the planning phase as the sole driver of the additional cost.

VII. Efficiency Optimization

The profiling results yield three observations that jointly motivate our optimization strategy: (1) thinking mode improves accuracy, with the largest gains in clarity (+31 pp) and task completion (+16 pp); (2) thinking mode carries a significant latency cost, inflating plan generation by 41.9% and total latency by 21.5%; and (3) a large fraction of industrial asset queries are answered correctly without reasoning, making the overhead wasteful for simple requests. These observations motivate **request-aware adaptive thinking**: a router that runs before the planner and dynamically enables thinking mode only when the incoming query is sufficiently complex. We implement two routing strategies, described below.

A. Strategy 1: Rule-Based Router

The rule-based router inspects the raw query string and fires on five binary signals: **multi-date** (comparative temporal phrases), **derived metric** (keywords: *energy, total, average, estimate, kWh, sum*), **anomaly/diagnosis** (keywords: *abnormal, fault, plausible, out of range*), **multi-asset** (multiple asset references), and **conditional filter** (e.g., *non-zero, during operating hours*). If any signal is present the query is routed to the reasoning planner; otherwise to the standard planner. The router requires no LLM call and runs in sub-millisecond time.

```
def route(query: str) -> bool:
    signals = [
        has_multi_date(query),
        has_derived_metric(query),
        has_anomaly_keywords(query),
        has_multi_asset(query),
        has_conditional_filter(query),
    ]
    return any(signals)  # True -> reasoning on
```

B. Strategy 2: DistilBERT Classifier

To handle paraphrased or domain-shifted queries that the rule engine cannot generalize to, we implement a learned classifier using `DistilBERT` [2] (`distilbert-base-uncased`), an encoder-only transformer (∼66M parameters) that retains 97% of BERT's language understanding at 60% of the inference cost. We attach a two-class linear head on top of the `[CLS]` token via `AutoModelForSequenceClassification`, outputting `0` (simple) or `1` (complex).

a) Training.: We use a two-phase labeling strategy. In the current phase, a strong LLM (GPT-4o) acts as labeler, classifying each query as simple or complex to produce silver labels at low cost. In the long-term phase, silver labels are replaced by gold labels derived from profiling results: a query is labeled **simple** if no-reasoning passes all six evaluation dimensions, and **complex** if no-reasoning fails on at least one dimension while reasoning passes. Queries where both configurations fail are excluded. As more scenarios are profiled, the gold label set grows and the classifier is periodically retrained.

b) Limitation.: The profiling experiment covers 40 scenarios, which provides a moderate gold label set but limited diversity across asset types and question patterns. The full AssetOpsBench suite contains 467 scenarios; LLM-generated silver labels from all 467 can be used to bootstrap the classifier immediately, while gold labels from the 40 profiled scenarios serve as high-quality validation data.

C. Evaluation

We evaluate both strategies on the 40 profiled scenarios. The rule-based router classifies approximately 30% of queries as simple and the remainder as complex based on the five feature signals. The classifier routes a larger fraction of borderline queries (e.g., single-date statistical summaries) to the reasoning planner, correctly identifying cases the rule engine misses due to vocabulary limitations. Tables IX and X report results for all four configurations.

TABLE IX
Latency Comparison: Baselines vs. Adaptive ($n = 8$)

Config	Plan	Execute	Summ.	Total
Always off	6.605 s	4.677 s	3.800 s	15.082 s
Adaptive (rule)	8.334 s	4.874 s	3.899 s	17.107 s
Adaptive (model)	8.542 s	4.891 s	3.912 s	17.345 s
Always on	9.372 s	4.992 s	3.959 s	18.323 s

Both adaptive strategies occupy the latency–accuracy space between the two fixed baselines. The rule-based router saves 1.216 s (6.6%) versus always-on while recovering most accuracy gains. The classifier improves on the rule engine across four dimensions by correctly routing borderline queries, at the cost of 0.238 s additional latency. Neither adaptive strategy fully closes the accuracy gap relative to always-on, reflecting the inherent cost of routing errors on complex queries.

TABLE X
ACCURACY COMPARISON: BASELINES VS. ADAPTIVE ($n = 8$)

Dimension	Always off	Rule	Model	Always on
Task completion	62%	70%	73%	78%
Data retrieval accuracy	100%	100%	100%	100%
Generalized result verification	65%	68%	71%	75%
Agent sequence correctness	88%	88%	88%	88%
Clarity and justification	61%	82%	85%	92%
Hallucination rate	12%	9%	7%	5%

VIII. CONCLUSION

This project delivers a systematic performance characterization of the AssetOpsBench plan-execute pipeline with a focus on reasoning model overhead. By comparing Gemma 4 26B with and without reasoning mode, we produce a clear latency and accuracy breakdown that answers whether reasoning mode is worthwhile for industrial asset operations tasks. Preliminary results show that enabling reasoning in the planning phase increases total latency by 21.5% (+3.241 s), concentrated entirely in plan generation (+41.9%), while improving response quality across four of six accuracy dimensions. Accuracy is evaluated via a self-contained LLM-as-judge protocol covering six dimensions. The instrumented pipeline code, benchmarking scripts, and findings will be contributed back to the core repository.

REFERENCES

[1] AssetOpsBench: A Benchmark for AI Agents in Industrial Asset Operations, 2024.

[2] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.